Module 2 - Automating Security Tasks with Python

2.1 - Working with Operating System Commands

Overview

This topic explores in depth how to use Python to interact with operating system (OS) commands, which is foundational for building automated defensive security tools. Mastery of this concept allows security analysts to automate diagnostics, collect forensic data, perform continuous monitoring, and integrate command-line tools within broader detection and response systems.

We will cover:

- Theoretical foundation
- `os.system()` vs `subprocess.run()`
- Capturing and processing outputs
- Handling interactive commands
- Real-time output streaming
- Error handling and timeouts
- Cross-platform considerations
- Command injection and input sanitization
- Use cases in security automation
- Real-world examples and a hands-on exercise

---

1. Why Automate OS Commands in Defensive Security?

In a defensive security environment such as a SOC, analysts frequently rely on OS utilities to gather system and network information. These include:

- `ping` : check network connectivity
- `netstat` : identify listening ports and connections
- `ipconfig` / `ifconfig` : retrieve interface configurations
- `whois` : resolve IP/domain ownership and metadata
- `ps` / `tasklist` : enumerate running processes
- `df` / `wmic` : gather disk/memory usage

These tools are highly effective when executed manually, but manual analysis does not scale. By using Python, you can:

- Automate and repeat checks across hundreds of systems
- Capture outputs and parse them for alerting or visualization
- Schedule recurring checks and generate periodic reports
- Chain multiple commands and enrich results with context

---

2. Methods to Execute Commands in Python

2.1 Using `os.system()`

A legacy but still commonly used function for simple command execution.

```
import os
os.system("ping -c 4 google.com")  # On Linux/macOS
# Windows: os.system("ping -n 4 google.com")
```

Pros:

- Quick and easy for testing
- Suitable for one-off tasks

Cons:

- Does not return command output
- Security risks with unsanitized input
- Limited error handling

Use this method only when output isn't needed or in trusted scripts.

2.2 Using `subprocess.run()`

This is the recommended modern approach.

```
import subprocess

result = subprocess.run(["ping", "-c", "4", "8.8.8.8"], capture_output=True, text=True)

print("Exit Code:", result.returncode)
print("Output:", result.stdout)
print("Errors:", result.stderr)
```

Advantages:

- Structured access to stdout, stderr, return code
- Built-in support for timeouts and shell isolation
- Enables programmatic analysis of command results

Use `text=True` to automatically decode byte output into strings.

---

3. Real-Time Output with `subprocess.Popen`

For long-running or streaming commands, you can use `Popen`:

```
import subprocess

process = subprocess.Popen(["ping", "-c", "5", "google.com"], stdout=subprocess.PIPE, text=True)

for line in process.stdout:
    print(">>>", line.strip())

process.wait()
```

This is particularly useful for:

- Logging live responses from network diagnostics
- Building TUI-based monitors or dashboards
- Streaming logs or events to SIEM-like pipelines

---

4. Handling Interactive Tools

Some tools expect user input (e.g., answering prompts). Use `communicate()`:

```
import subprocess

proc = subprocess.Popen(["some_interactive_command"], stdin=subprocess.PIPE, stdout=subprocess.PIPE, text=True)
stdout, stderr = proc.communicate("yes\n")

print(stdout)
```

5. Error Handling and Timeouts

Security tools must be robust. Always implement proper error control.

Example: Timeout Protection

```
import subprocess

try:
    subprocess.run(["sleep", "10"], timeout=3)
except subprocess.TimeoutExpired:
    print("Command timed out.")
```

Example: Exit Code Checks

```
result = subprocess.run(["ls", "/nonexistent"], capture_output=True, text=True)

if result.returncode != 0:
    print("Error occurred:", result.stderr)
```

---

6. Parsing Command Output

After executing a command, parsing its output is essential for automation.

Example: Extract WHOIS Info

```
import subprocess
import re
```

```python
domain = "example.com"
result = subprocess.run(["whois", domain], capture_output=True, text=True)

patterns = {
    "Registrar": r"Registrar:\s*(.+)",
    "Created": r"Creation Date:\s*(.+)",
    "Expires": r"Expiry Date:\s*(.+)"
}

for key, regex in patterns.items():
    match = re.search(regex, result.stdout)
    if match:
        print(f"{key}: {match.group(1)}")
```

You can adapt this technique for any structured or semi-structured CLI output.

---

7. Cross-Platform Compatibility

OS command syntax differs between systems. Use `platform.system()` to adapt:

```python
import platform
import subprocess

if platform.system() == "Windows":
    cmd = ["ipconfig"]
else:
    cmd = ["ifconfig"]

subprocess.run(cmd)
```

Avoid hardcoding OS-specific commands in cross-platform tools. Abstract them into helper functions.

---

8. Security Considerations

Avoid Shell Injection

Bad practice (dangerous with user input):

```
subprocess.run(f"ping {user_input}", shell=True)
```

Preferred:

```
subprocess.run(["ping", user_input])
```

- Always validate input (IP/domain formats)
- Never pass unchecked user data to `shell=True`
- Avoid using `shell=True` unless required by syntax (e.g., redirection, pipes)

---

9. Use Case: Host Availability and Domain Intel Script

```
import subprocess
import json

domain = "scanme.nmap.org"

# Ping check
ping = subprocess.run(["ping", "-c", "3", domain], capture_output=True, text=True)
```

```python
reachable = "0% packet loss" in ping.stdout

# WHOIS data
whois = subprocess.run(["whois", domain], capture_output=True, text=True)
whois_data = {}
for line in whois.stdout.splitlines():
    for field in ["Registrar:", "Creation Date:", "Expiry Date:"]:
        if line.startswith(field):
            whois_data[field] = line.split(":", 1)[-1].strip()

report = {
    "domain": domain,
    "reachable": reachable,
    "whois": whois_data
}

print(json.dumps(report, indent=2))
```

This script integrates multiple command-line tools into a structured report.

---

10. Practical Exercise

Write a Python program that:

1. Prompts the user for a domain name
2. Validates the domain format
3. Runs a ping check and WHOIS lookup
4. Parses and displays key metadata
5. Outputs the results as a well-formatted JSON file
6. Includes exception handling for timeouts and command errors

Bonus:

- Adapt it to work on both Windows and Linux/macOS
- Add logging to a file

2.2 - Automating Reconnaissance and Scanning with Python

Objective

The goal of this topic is to teach how to automate reconnaissance and scanning operations using Python. These capabilities are fundamental in both blue team and red team roles, especially in environments where you must monitor assets, assess vulnerabilities, or map services continuously.

We will focus on:

- Calling external scanners from Python (e.g., `nmap`)
- Parsing their outputs (e.g., XML, stdout)
- Performing basic port scanning with sockets
- Enriching reconnaissance with context (IP info, banners)
- Handling errors, timeouts, and OS compatibility
- Using APIs when appropriate for speed and accuracy

---

1. Why Automate Reconnaissance?

Reconnaissance is the foundation of all security analysis. For defenders, this includes:

- Asset discovery (finding what's online)
- Port and service enumeration
- Technology fingerprinting

- Monitoring changes in exposed services
- Detecting rogue or unpatched systems

By automating recon, we can schedule scans, enrich results, and react to exposure faster.

---

2. Executing External Scanners: Nmap Example

2.1 Running Nmap from Python

Nmap is one of the most widely used network scanners. You can execute it directly from Python using the `subprocess` module.

```
import subprocess

# Define target and ports
target = "scanme.nmap.org"
ports = "22,80,443"

# Run Nmap with common flags:
# -Pn: skip ping discovery
# -sS: TCP SYN scan (stealth)
# -p: specify ports
command = ["nmap", "-Pn", "-sS", "-p", ports, target]

# Run and capture the output
result = subprocess.run(command, capture_output=True, text=True)

# Print the raw Nmap output
print(result.stdout)
```

Explanation:

- `subprocess.run()` safely invokes the Nmap command.
- Using list format avoids shell injection risks.
- Capturing output allows us to parse and automate decisions.

---

3. Parsing Nmap Output (Structured XML)

Nmap supports structured output via the `-oX` flag, which is ideal for automation.

Step 1: Run the scan and save XML

```
nmap -p 22,80,443 -Pn scanme.nmap.org -oX output.xml
```

Step 2: Parse the XML with `xml.etree.ElementTree`

```
import xml.etree.ElementTree as ET

# Load and parse the XML file
tree = ET.parse("output.xml")
root = tree.getroot()

# Iterate over host blocks
for host in root.findall("host"):
    ip = host.find("address").get("addr")
    print(f"Host: {ip}")

    # Get open ports
    for port in host.findall("ports/port"):
        port_id = port.get("portid")
        protocol = port.get("protocol")
        state = port.find("state").get("state")
```

```
        service = port.find("service").get("name", "unknown")
        print(f" - Port {port_id}/{protocol}: {state} ({service})")
```

Explanation:

- `ElementTree` parses the XML into nested Python objects.
- We extract the host IP and loop over each port.
- This allows structured reporting and integration with dashboards or alerts.

---

4. Manual Port Scanning with Sockets

If Nmap is unavailable or you need a quick check, Python's `socket` module can scan ports directly.

```
import socket

def scan_ports(target, ports):
    open_ports = []
    for port in ports:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(1)  # Short timeout for speed
        result = sock.connect_ex((target, port))  # 0 means open
        if result == 0:
            open_ports.append(port)
        sock.close()
    return open_ports

# Example usage
target_host = "scanme.nmap.org"
ports_to_test = [22, 80, 443, 8080]
```

```
open_found = scan_ports(target_host, ports_to_test)
print(f"Open ports on {target_host}: {open_found}")
```

Explanation:

- `connect_ex()` returns 0 if the port is open.
- We set a short timeout to avoid blocking.
- You can expand this to include banner grabbing or logging.

---

5. Adding IP Context with APIs (Optional)

To enrich scan results, you can query public IP intelligence APIs.

Using `ipinfo.io` API:

```
import requests

ip = "8.8.8.8"
response = requests.get(f"https://ipinfo.io/{ip}/json")

if response.status_code == 200:
    data = response.json()
    print("IP Info:")
    print(f" - City: {data.get('city')}")
    print(f" - Org: {data.get('org')}")
    print(f" - ASN: {data.get('asn', {}).get('asn', 'N/A')}")
```

This is useful for determining whether an exposed IP belongs to a cloud provider, CDN, or suspicious entity.

---

6. Combining Recon and Scan in One Script

```python
import subprocess
import socket
import xml.etree.ElementTree as ET

def run_nmap(target, ports):
    xml_out = "temp.xml"
    command = ["nmap", "-Pn", "-sS", "-p", ports, target, "-oX", xml_out]
    subprocess.run(command)
    return xml_out

def parse_nmap(xml_file):
    tree = ET.parse(xml_file)
    root = tree.getroot()
    report = []

    for host in root.findall("host"):
        ip = host.find("address").get("addr")
        for port in host.findall("ports/port"):
            port_id = port.get("portid")
            state = port.find("state").get("state")
            service = port.find("service").get("name", "unknown")
            report.append((ip, port_id, state, service))
    return report

def quick_socket_scan(target, ports):
    found = []
    for port in ports:
        s = socket.socket()
        s.settimeout(1)
        if s.connect_ex((target, port)) == 0:
            found.append(port)
```

```
        s.close()
    return found

# Run full recon
host = "scanme.nmap.org"
nmap_result = run_nmap(host, "22,80,443")
parsed_data = parse_nmap(nmap_result)
print("[+] Nmap XML Parsed:")
for row in parsed_data:
    print(row)

print("[+] Socket scan result:", quick_socket_scan(host, [22, 80, 443]))
```

---

7. Logging and Alerting

You can write all this data to files, databases, or send alerts via messaging apps like Telegram or Slack. Add timestamped logging using the `logging` module.

---

8. Defensive Use Cases

- **Change Monitoring**: Detect if new ports are opened unexpectedly
- **Asset Inventory**: Automatically scan IP ranges in your organization
- **Vulnerability Scope Validation**: Check which services are exposed before and after patch cycles
- **Threat Detection**: Alert on rogue services (e.g., unencrypted RDP)

---

9. Exercise (Hands-on)

Write a script that:

1. Accepts an IP address or domain as input.
2. Performs a TCP SYN scan on ports 20-1024 using Nmap.
3. Parses the output using XML.
4. Runs a socket scan on the same ports as validation.
5. Logs all results to a local file with timestamps.
6. (Optional) Sends an alert via Telegram if port 3389 or 23 is open.

Automating reconnaissance and scanning in Python gives defenders visibility, speed, and precision. You can proactively identify exposed assets, monitor environments for misconfigurations, and build context around threats. By integrating scanners like Nmap and combining them with native capabilities like `socket` and APIs, your tooling becomes robust, extensible, and enterprise-ready.

2.3 - Manual Port Scanning with Python Sockets

Objective

The goal of this topic is to provide a thorough understanding of how to perform manual port scanning using Python's `socket` module. This method offers complete control over how ports are tested, how services are identified, and how results are handled and interpreted. Manual scanning is useful in low-level scripting, learning TCP/IP behavior, and developing lightweight tools for asset discovery and monitoring.

---

1. Introduction to Manual Port Scanning

Port scanning is the process of probing a host to determine which network ports are open and listening for connections. Each open port represents a potential service running on the host, which could be vulnerable, misconfigured, or simply unexpected.

Whereas tools like Nmap abstract away the underlying connection logic, manual port scanning allows you to:

- Build custom scanners tailored to specific environments
- Understand low-level behavior and response timing
- Integrate logic to fingerprint banners or analyze anomalies
- Develop portable scanning utilities without dependencies

---

2. The Python `socket` Module: Fundamentals

Python's built-in `socket` module provides an interface to the BSD socket API. It allows you to:

- Create TCP or UDP connections
- Send and receive raw data
- Handle timeouts and connection exceptions

Basic TCP Socket Example:

```python
import socket

# Create a TCP socket object
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Set timeout to avoid hanging on unresponsive hosts
sock.settimeout(2)

# Try connecting to a host and port
result = sock.connect_ex(("example.com", 80))

if result == 0:
    print("Port 80 is open")
else:
    print("Port 80 is closed or filtered")
```

```
sock.close()
```

Explanation:

- `socket.AF_INET` defines the IPv4 address family.
- `socket.SOCK_STREAM` creates a TCP connection.
- `connect_ex()` attempts to connect and returns `0` on success or an error code.
- Timeouts prevent scripts from hanging indefinitely.

---

3. Writing a Full Manual Port Scanner

We can now expand the logic into a full scanner that tests multiple ports and handles common edge cases.

Basic Multi-Port Scanner:

```
import socket

def scan_ports(target, port_list, timeout=1):
    open_ports = []

    for port in port_list:
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(timeout)

            result = sock.connect_ex((target, port))

            if result == 0:
                print(f"[+] Port {port} is open")
```

```
                open_ports.append(port)
            else:
                print(f"[-] Port {port} is closed or filtered")

            sock.close()

        except Exception as e:
            print(f"[!] Error scanning port {port}: {e}")

    return open_ports

# Example usage
target_host = "scanme.nmap.org"
ports_to_scan = list(range(20, 1025))  # Common ports

found_ports = scan_ports(target_host, ports_to_scan)
print("Open ports found:", found_ports)
```

Explanation:

- Loops through a list of ports.
- Each port is tested individually using a short-lived TCP socket.
- Results are appended to a list for later use or reporting.

---

4. Enhancing the Scanner: Threading for Speed

Manual scanning over many ports can be slow due to timeouts. We can use threading to scan ports concurrently.

Threaded Scanner Example:

```python
import socket
import threading

open_ports = []
lock = threading.Lock()

def scan_port(target, port):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(1)
        result = sock.connect_ex((target, port))
        sock.close()

        if result == 0:
            with lock:
                print(f"[+] Port {port} is open")
                open_ports.append(port)
    except:
        pass

def threaded_scan(target, ports):
    threads = []

    for port in ports:
        t = threading.Thread(target=scan_port, args=(target, port))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    return open_ports
```

```python
# Example usage
target = "scanme.nmap.org"
ports = range(1, 1025)

results = threaded_scan(target, ports)
print("Scan complete. Open ports:", results)
```

Threading Benefits:

- Speeds up scans by running multiple checks in parallel.
- Shared data (`open_ports`) is protected by a lock for thread safety.

---

5. Banner Grabbing

Some services respond with metadata (e.g., a web server header). We can connect and read the initial response to identify the service.

Example:

```python
def grab_banner(ip, port):
    try:
        s = socket.socket()
        s.settimeout(2)
        s.connect((ip, port))
        banner = s.recv(1024)
        s.close()
        return banner.decode(errors='ignore')
    except:
        return None

# Use banner grabbing after confirming port is open
ip = "scanme.nmap.org"
```

```python
for port in [21, 22, 80]:
    banner = grab_banner(ip, port)
    if banner:
        print(f"Banner from port {port}:\n{banner}")
```

Use Cases:

- Identify service versions (Apache, SSH, etc.)
- Detect honeypots or fake services
- Enrich vulnerability scans with context

---

6. Exception Handling and Resilience

You must anticipate edge cases:

- Host unreachable
- Network errors
- Invalid IP formats
- Firewall interference

Best Practices:

- Always close sockets (use `with` or `try/finally`)
- Catch specific exceptions (`socket.timeout`, `socket.gaierror`)
- Respect timing policies (avoid flooding production systems)
- Support input validation

---

7. Logging and JSON Export

Add logging or export to JSON for later analysis.

```
import json
from datetime import datetime

def save_results(host, open_ports):
    report = {
        "host": host,
        "timestamp": datetime.utcnow().isoformat(),
        "open_ports": open_ports
    }

    with open("scan_results.json", "w") as f:
        json.dump(report, f, indent=2)

# Save the output
save_results("scanme.nmap.org", [22, 80, 443])
```

---

8. Cross-Platform Considerations

- `socket` works identically on Windows and Linux.
- On Windows, certain ports (like 135, 445) may behave differently due to OS-level handling.
- If using `os.system()` to resolve IPs or perform pre-checks (e.g., `ping`), ensure platform compatibility.

---

9. Use Cases in Defensive Security

Manual port scanning scripts are valuable for:

- **Local agent tools** that validate perimeter exposure
- **Internal network mapping** during blue team assessments
- **Security control validation**, e.g., verifying that port blocks are enforced
- **Incident response**, e.g., checking lateral movement paths
- **Asset inventory updates** for systems not yet registered

---

10. Exercise

Write a complete script that:

1. Accepts a domain or IP from the user.
2. Resolves the domain to an IP.
3. Performs a threaded port scan on ports 1–1024.
4. Grabs banners from open ports.
5. Exports the data in a structured JSON format.
6. Handles all possible exceptions gracefully.

Bonus:

- Add `argparse` support for CLI usage
- Store results in a local SQLite database

2.4 - Interacting with APIs and External Services using Python

Objective

This topic will teach you how to integrate Python with external APIs and services to enhance your defensive security workflows. By leveraging APIs, you can enrich alerts, automate threat intelligence lookups, monitor external attack surfaces, and perform real-time enrichment of logs or events.

We will cover:

- The fundamentals of HTTP requests with `requests`
- Authentication with API tokens
- GET vs POST methods
- JSON data handling
- Error and rate-limit handling
- Consuming public APIs (e.g., IP geolocation, threat intel, hash scanning)
- Building a modular API client
- Logging and alerting integration

---

1. Introduction: Why Use APIs in Defensive Security?

In modern security operations, external data sources are critical. APIs allow your scripts and tools to:

- Enrich IOCs (IP addresses, domains, hashes) with context
- Query reputation databases (VirusTotal, AbuseIPDB, Shodan, etc.)
- Detect exposed services or credentials
- Automate decision-making based on external threat feeds

These actions help reduce false positives, prioritize alerts, and identify high-risk assets.

---

2. Python and HTTP APIs: Using the `requests` Library

Python's `requests` library is the de facto standard for HTTP interaction.

Installing `requests` (if not already):

```
pip install requests
```

---

3. Basic API Call: IPInfo.io Example

Code:

```
import requests

ip = "8.8.8.8"
url = f"https://ipinfo.io/{ip}/json"

response = requests.get(url)

# Check if the request was successful
if response.status_code == 200:
    data = response.json()
    print(f"IP: {data.get('ip')}")
    print(f"City: {data.get('city')}")
    print(f"Organization: {data.get('org')}")
    print(f"Location: {data.get('loc')}")
else:
    print(f"Error {response.status_code}: {response.text}")
```

Explanation:

- `requests.get()` sends an HTTP GET request.

- `response.json()` parses the returned JSON body into a Python dictionary.
- Use `.get()` for safe access (won't throw `KeyError`).

---

4. Adding Authentication Headers

Most APIs require an API key or token passed as a header or query parameter.

Example: VirusTotal File Hash Scan

```
import requests

API_KEY = "your_api_key_here"
file_hash = "44d88612fea8a8f36de82e1278abb02f"  # EICAR test file
url = f"https://www.virustotal.com/api/v3/files/{file_hash}"

headers = {
    "x-apikey": API_KEY
}

response = requests.get(url, headers=headers)

if response.status_code == 200:
    result = response.json()
    stats = result['data']['attributes']['last_analysis_stats']
    print("Malicious:", stats['malicious'])
    print("Suspicious:", stats['suspicious'])
else:
    print("Error:", response.status_code)
```

Best Practices:

- Never hardcode API keys. Use `.env` or secure vaults.
- Always handle 4xx/5xx errors gracefully.
- Implement rate-limit backoff if needed.

---

5. Working with POST Requests

Some APIs (e.g., submission APIs) require POST requests with data.

Example: POST with JSON Body

```
import requests

url = "https://httpbin.org/post"

payload = {
    "username": "admin",
    "action": "login"
}

response = requests.post(url, json=payload)

print("Status:", response.status_code)
print("Server Response:", response.json())
```

- `json=` auto-converts Python dict to JSON and sets content type
- For raw body or form-encoded data, use `data=` or `files=`

---

6. Handling Errors, Timeouts, and Retries

You must anticipate problems with remote services.

Example with Try-Except and Timeout

```
import requests

try:
    r = requests.get("https://ipinfo.io/8.8.8.8/json", timeout=5)
    r.raise_for_status()  # Raise error if response is 4xx or 5xx
    data = r.json()
    print(data)
except requests.exceptions.HTTPError as err:
    print("HTTP error:", err)
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.RequestException as e:
    print("General error:", e)
```

Optional: Retry with `backoff` or `tenacity` libraries

These add exponential retry behavior automatically.

---

7. Modularizing API Logic

Instead of hardcoding API logic, create reusable client classes.

Example: IPInfoClient Class

```python
import requests

class IPInfoClient:
    def __init__(self, token=None):
        self.base_url = "https://ipinfo.io"
        self.token = token

    def lookup(self, ip):
        url = f"{self.base_url}/{ip}/json"
        params = {"token": self.token} if self.token else {}
        try:
            response = requests.get(url, params=params, timeout=5)
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e:
            print(f"[!] API error: {e}")
            return None

# Example usage
client = IPInfoClient()
info = client.lookup("8.8.8.8")
if info:
    print(info.get("org"), info.get("city"))
```

Benefits of Modularization:

- Reuse across multiple scripts
- Easier testing and maintenance
- Encapsulates error handling

---

8. Real-World Use Cases in Security Operations

IOC Enrichment

Use APIs to automatically enrich logs with:

- ASN ownership
- Country
- Provider
- Threat reputation

Automated Analysis Pipelines

Upload files or hashes to VirusTotal or HybridAnalysis and store/report results automatically.

Blocklist Integration

Check whether an IP or domain appears in threat feeds, and block if necessary.

Alert Prioritization

Only escalate if the IP appears in multiple threat sources or has a “malicious” rating.

---

9. JSON Handling and Integration

API responses are often in JSON. Use standard techniques to traverse and process them.

```
# Sample nested access
engine_results = data['data']['attributes']['last_analysis_results']
```

```
for engine, result in engine_results.items():
    print(f"{engine}: {result['category']} - {result['result']}")
```

Use `json.dumps(obj, indent=2)` to pretty-print JSON to logs or reports.

---

10. Logging API Results

```
import logging

logging.basicConfig(filename="api.log", level=logging.INFO)

def log_ip_info(data):
    ip = data.get("ip")
    city = data.get("city")
    org = data.get("org")
    logging.info(f"{ip} - {city} - {org}")
```

Integrate API data into your security logs or case management system.

---

11. Exercise

Create a Python script that:

1. Accepts an IP from the user.
2. Looks up the IP using both:
   - IPInfo.io
   - AbuseIPDB (optional)

3. Checks for threat reputation and location.
4. Outputs the information in a structured table.
5. Logs results to a local file.
6. Implements error handling for network/API issues.

Bonus:

- Implement CLI flags with `argparse`
- Schedule it to run hourly using `schedule` or cron
- Output to CSV or JSON file

2.5 - Filesystem Monitoring and Data Collection

Objective

In this topic, you'll learn how to monitor the filesystem, collect metadata, calculate cryptographic hashes, and identify suspicious changes or artifacts. This is especially useful for:

- Digital forensics and incident response (DFIR)
- Malware persistence tracking
- File integrity monitoring (FIM)
- Detection of unauthorized file creation or modification
- Security auditing and automation in SOCs

We'll cover:

- File and directory traversal
- Metadata extraction (timestamps, permissions, sizes)
- Hashing files (MD5, SHA256, etc.)
- Change detection using baseline comparison
- Real-time monitoring with watchdog

- Logging and alerting

---

1. Introduction: Why Monitor the Filesystem?

Filesystem monitoring allows defenders to detect:

- Web shell uploads
- Persistence mechanisms (autoruns, cronjobs, services)
- Ransomware behavior (mass encryption/modification)
- File tampering and defacements
- Evidence of lateral movement (malware dropped files)

Manual review of files is not scalable. Automation with Python enables real-time or scheduled scanning, alerting, and integration with SIEMs.

---

2. Traversing the Filesystem

The first step in monitoring is scanning the file tree.

Example: Recursively Listing Files

```
import os

def list_all_files(base_path):
    for root, dirs, files in os.walk(base_path):
        for name in files:
            full_path = os.path.join(root, name)
            print(full_path)
```

```
# Example usage
list_all_files("/var/log")
```

Explanation:

- `os.walk()` traverses a directory tree.
- Yields a tuple of `(root, subdirs, files)` at each level.
- Paths are joined safely using `os.path.join()`.

---

3. Extracting File Metadata

Python can retrieve information such as size, modification time, creation time, and permissions.

Example: Getting Metadata for a File

```
import os
import time

def get_file_metadata(path):
    try:
        stat = os.stat(path)
        print(f"File: {path}")
        print(f"  Size: {stat.st_size} bytes")
        print(f"  Last Modified: {time.ctime(stat.st_mtime)}")
        print(f"  Created: {time.ctime(stat.st_ctime)}")
        print(f"  Permissions: {oct(stat.st_mode)[-3:]}")
    except FileNotFoundError:
        print(f"[!] File not found: {path}")
```

```
get_file_metadata("/etc/passwd")
```

Note:

- On Windows, `ctime` = creation time
- On Linux, `ctime` = last metadata change

---

4. Hashing Files for Integrity

Cryptographic hashes allow us to detect file changes precisely.

Example: Hash a File with SHA-256

```
import hashlib

def hash_file(path):
    try:
        with open(path, "rb") as f:
            data = f.read()
            return hashlib.sha256(data).hexdigest()
    except Exception as e:
        return f"[ERROR] {e}"

print(hash_file("/etc/hosts"))
```

Variants:

- Use `hashlib.md5()` for speed (less secure)
- Use `hashlib.sha256()` for security-critical tasks

- Hashing large files? Read in chunks (below)

---

5. Efficient Hashing with Buffered Reading

For large files (e.g., log archives, ISOs), avoid loading the whole file into memory.

```
def hash_large_file(path, algo="sha256", buffer_size=65536):
    h = getattr(hashlib, algo)()
    try:
        with open(path, "rb") as f:
            while chunk := f.read(buffer_size):
                h.update(chunk)
        return h.hexdigest()
    except Exception as e:
        return f"[ERROR] {e}"

print(hash_large_file("/var/log/syslog"))
```

Explanation:

- Buffered reads minimize memory usage.
- Works with files of any size.

---

6. Creating a Baseline for File Integrity

We can create a snapshot (baseline) of all files and their hashes, then compare later to detect tampering.

Step 1: Create Baseline

```python
import json

def create_baseline(directory, output_file):
    baseline = {}
    for root, _, files in os.walk(directory):
        for f in files:
            path = os.path.join(root, f)
            baseline[path] = hash_large_file(path)

    with open(output_file, "w") as out:
        json.dump(baseline, out, indent=2)

create_baseline("/etc", "baseline.json")
```

Step 2: Compare to Baseline

```python
def compare_baseline(baseline_file):
    with open(baseline_file, "r") as f:
        old = json.load(f)

    for path, old_hash in old.items():
        new_hash = hash_large_file(path)
        if new_hash != old_hash:
            print(f"[MODIFIED] {path}")

compare_baseline("baseline.json")
```

Optional Enhancements:

- Track added/removed files
- Log results with timestamps

- Send alerts on changes

---

7. Real-Time Monitoring with `watchdog`

For real-time monitoring, use the `watchdog` module, which listens to filesystem events using OS-level APIs.

Installation:

```
pip install watchdog
```

Example: Watch a Directory

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import time

class MonitorHandler(FileSystemEventHandler):
    def on_modified(self, event):
        print(f"[Modified] {event.src_path}")

    def on_created(self, event):
        print(f"[Created] {event.src_path}")

    def on_deleted(self, event):
        print(f"[Deleted] {event.src_path}")

def watch_directory(path):
    observer = Observer()
    handler = MonitorHandler()
    observer.schedule(handler, path=path, recursive=True)
```

```
    observer.start()
    print(f"[INFO] Watching {path}... Press CTRL+C to exit.")

    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
    observer.join()

# Example usage
watch_directory("/tmp")
```

Use Cases:

- Detect web shells in webroot directories
- Watch critical folders for ransomware activity
- Monitor logs for changes outside business hours

---

8. Logging and Alerting

Logging Detected Changes:

```
import logging

logging.basicConfig(filename="fs_monitor.log", level=logging.INFO)

def log_change(event_type, path):
    logging.info(f"{event_type}: {path}")
```

Integrate with Alerting:

- Email via `smtplib`
- Telegram/Slack via webhook
- REST API to forward events to SIEM

---

9. Handling Exclusions and Patterns

Often, you want to ignore certain file types or directories.

Example: Exclude `.log` files

```
if not path.endswith(".log"):
    hash_file(path)
```

Or use `fnmatch`:

```
from fnmatch import fnmatch

if not fnmatch(path, "*.log"):
    # Process file
```

---

10. Use Cases in Defensive Security

- **File Integrity Monitoring (FIM):** Detect unauthorized modification of config files or binaries.
- **Persistence Detection:** Alert if `.bashrc`, cron, or startup folders are modified.

- **Data Exfiltration Tracking:** Monitor for creation of large `.zip`, `.rar`, `.db` files in user directories.
- **Webshell Hunting:** Watch `uploads/` or `htdocs/` for new `.php` files.

---

11. Exercise

Build a complete script that:

1. Accepts a directory path from the user
2. Scans all files, logs their metadata and SHA-256 hash
3. Stores results in a JSON baseline file
4. Offers a `--compare` mode to check against the baseline
5. Includes a `--watch` mode using `watchdog` for real-time monitoring
6. Logs all activity and optionally alerts via terminal or file

Bonus:

- Add file size thresholds for alerting (e.g., >10MB in `/etc`)
- Track file owner/group via `os.stat().st_uid` and `pwd.getpwuid()`

2.6 - Scheduling and Continuous Monitoring

Objective

The objective of this topic is to empower learners to build reliable, automated monitoring systems using Python. You'll learn how to:

- Schedule periodic tasks (like filesystem scans, API checks, alerts)
- Automate monitoring routines with various Python tools
- Utilize time-based triggers (cron-like behavior)
- Create persistent daemons or long-running scripts

- Combine this with logging, alerting, and filesystem monitoring

This is a critical step for security automation: being able to continuously watch systems, detect anomalies, and react — all without manual intervention.

---

1. Introduction to Task Scheduling

In defensive security, many tasks must run repeatedly:

- Periodic integrity checks
- Daily backups and scans
- Scheduled API queries (e.g., IP reputation)
- Log scraping and anomaly detection

Python doesn't have a built-in "cron" mechanism, but it integrates well with schedulers and offers libraries like `schedule`, `threading`, `time`, and `APScheduler`.

---

2. The `time.sleep()` Loop – The Most Basic Scheduler

This is the simplest form of repeated execution.

Example: Run a Task Every 10 Seconds

```
import time

def job():
    print("Running task...")
```

```
while True:
    job()
    time.sleep(10)  # Sleep for 10 seconds before next execution
```

✅ Pros:

- Very simple
- No dependencies

❌ Cons:

- Doesn't run at exact clock times (just intervals)
- No task management or tracking

---

3. Using `schedule` Library – Human-Friendly Scheduling

The `schedule` library allows you to define jobs with human-readable syntax.

Installation:

```
pip install schedule
```

Example: Run a Job Every Minute

```
import schedule
import time

def check_filesystem():
    print("Scanning filesystem for changes...")
```

```
# Schedule it every 1 minute
schedule.every(1).minutes.do(check_filesystem)

# Run continuously
while True:
    schedule.run_pending()
    time.sleep(1)
```

🔍 Breakdown:

- `schedule.every(1).minutes.do(...)` : defines when to run
- `run_pending()` checks if a job is due
- `time.sleep(1)` avoids busy looping

Other scheduling options:

```
schedule.every().hour.do(job)
schedule.every().day.at("03:30").do(job)
schedule.every().monday.at("14:00").do(job)
```

---

4. Advanced Scheduling with APScheduler

`APScheduler` is a more powerful option that supports:

- Persistent jobs (backed by database)
- Cron-style expressions
- Background threads and process pools

Installation:

```
pip install apscheduler
```

Example: Cron-style Scheduling

```
from apscheduler.schedulers.background import BackgroundScheduler
import time

def task():
    print("Checking for suspicious file changes...")

scheduler = BackgroundScheduler()
scheduler.add_job(task, 'cron', hour=3, minute=0)  # Run daily at 3:00 AM
scheduler.start()

try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    scheduler.shutdown()
```

📌 Notes:

- `'cron'` trigger allows precise time rules
- Supports other trigger types: `'interval'`, `'date'`
- Can schedule multiple jobs simultaneously

---

5. Persistent Daemon for Continuous Monitoring

To continuously monitor and respond in real time (like a mini-agent), build a daemon using `threading` or `multiprocessing`.

Example: Watchdog + Threaded Job

```python
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import threading
import time

class FSHandler(FileSystemEventHandler):
    def on_modified(self, event):
        print(f"[Modified] {event.src_path}")

    def on_created(self, event):
        print(f"[Created] {event.src_path}")

def watch_folder(path):
    observer = Observer()
    observer.schedule(FSHandler(), path, recursive=True)
    observer.start()
    print(f"Watching {path}...")
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        observer.stop()
    observer.join()

def hourly_task():
    while True:
        print("[Scheduled] Running hourly task")
        time.sleep(3600)  # Every hour
```

```
# Start threads
threading.Thread(target=watch_folder, args=("/tmp",)).start()
threading.Thread(target=hourly_task).start()
```

✅ Use Case:

- One thread for real-time monitoring (e.g., file changes)
- Another thread for scheduled integrity checks

---

6. Combining Logging and Alerting

Use the `logging` module to capture task results.

Example: Log Changes Detected

```
import logging

logging.basicConfig(filename="monitoring.log", level=logging.INFO)

def log_event(msg):
    logging.info(f"{time.ctime()} | {msg}")

# In your task:
log_event("Found unexpected .exe in /tmp")
```

To alert:

- Use email (`smtplib`)
- Send HTTP requests (Slack, Telegram)
- Trigger webhook to external SIEM

7. Integration with System-Level Schedulers

If you don't want to rely on Python loops, schedule Python scripts with:

Linux Cron Example

```
crontab -e
# Run every day at 2 AM
0 2 * * * /usr/bin/python3 /opt/scripts/check_hashes.py
```

Windows Task Scheduler

- Create a task to run `python.exe` with your script as argument
- Set triggers: daily, weekly, at logon, etc.

---

8. Use Case: Scheduled Integrity Scan with Baseline Comparison

You can combine all the techniques to build a resilient monitoring system.

Simplified Overview:

1. Use `os.walk` + `hashlib` to scan files
2. Save hashes to `baseline.json`
3. Schedule comparison job every 4 hours
4. Use `watchdog` for real-time detection
5. Log all activity
6. Alert on anomalies

This provides both **interval-based detection** and **real-time reaction**.

---

9. Best Practices

- Use threading or multiprocessing to prevent blocking
- Keep logs clean and rotate them (e.g., `logging.handlers.RotatingFileHandler`)
- Limit scope of monitoring (exclude temp/log folders)
- Alert on anomalies, not everything (reduce noise)
- Test scheduling accuracy with simulated delays

---

10. Final Exercise

Build a Python-based monitoring suite that:

- Loads baseline from `baseline.json`
- Checks files every 2 hours using `schedule`
- Monitors `/var/www/` with `watchdog`
- Logs all changes to `monitoring.log`
- Sends alert if a `.php` or `.exe` file is created

Bonus:

- Add CLI interface to select `--rebuild-baseline` or `--monitor-mode`
- Package it as a service using `pyinstaller` or `systemd`

2.7 - Data Handling and Output Generation

Objective

This topic focuses on how to handle structured and unstructured data effectively using Python. Students will learn to:

- Parse input data (JSON, CSV, text logs)
- Store and manipulate security-related data
- Format outputs for reports and dashboards
- Export data to various formats (JSON, CSV, TXT, HTML)
- Prepare data for alerting, visualization, or external systems

Data handling and reporting are essential in cybersecurity—whether parsing SIEM logs, generating incident reports, or summarizing scan results. The clarity, structure, and reliability of your output directly impact the effectiveness of your response.

---

1. Reading and Parsing Data

1.1 Reading JSON Files

JSON is the most common format in cybersecurity tools and APIs.

```python
import json

with open("scan_result.json", "r") as f:
    data = json.load(f)

# Print a specific field
print("IP Address:", data["host"]["ip"])
```

Use Case: Reading the results from a vulnerability scanner or threat intelligence API.

1.2 Reading CSV Files

CSV files are common in logs, reports, and exported data.

```
import csv

with open("firewall_logs.csv", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"Source IP: {row['src_ip']}, Destination: {row['dst_ip']}")
```

Use Case: Parsing firewall logs for incident correlation.

1.3 Reading Plain Text or Log Files

Many tools output text logs that must be parsed.

```
with open("alerts.log") as f:
    for line in f:
        if "BLOCKED" in line:
            print("[!] Blocked traffic:", line.strip())
```

Use string search, regex, or split() for structured lines:

```
import re

line = "2025-06-14 12:01:23 BLOCKED 192.168.1.4 -> 10.0.0.1"
match = re.search(r"BLOCKED\s+(\S+)\s+->\s+(\S+)", line)
```

```
if match:
    print("Source:", match.group(1), "Target:", match.group(2))
```

---

2. Storing and Manipulating Data

You often need to transform and enrich raw data.

2.1 Data as Python Dictionaries and Lists

```
incident = {
    "id": "INC10293",
    "timestamp": "2025-06-14T13:45:00Z",
    "severity": "high",
    "affected_hosts": ["10.0.0.1", "10.0.0.2"],
}
incident["status"] = "open"  # Add new field
incident["affected_hosts"].append("10.0.0.3")  # Update list

print(json.dumps(incident, indent=2))  # Pretty-print
```

2.2 Normalizing and Flattening Nested Data

Many security APIs return deeply nested structures. Use functions to flatten or simplify them.

```
def flatten_event(event):
    return {
        "timestamp": event["meta"]["time"],
        "ip": event["network"]["ip_src"],
```

```
        "url": event.get("network", {}).get("http_request", {}).get("url", "N/A")
    }

# Example structure
raw_event = {
    "meta": {"time": "2025-06-14T13:00:00Z"},
    "network": {"ip_src": "192.168.1.3", "http_request": {"url": "http://malware.com"}}
}

print(flatten_event(raw_event))
```

---

3. Output Generation – Writing to Files

Python can export to many formats. This is critical for:

- Alerts
- Dashboards
- Compliance reports

3.1 Writing to JSON

```
result = {
    "host": "192.168.1.5",
    "status": "infected",
    "threat": "Ransomware",
    "timestamp": "2025-06-14T14:15:00Z"
}

with open("report.json", "w") as f:
    json.dump(result, f, indent=4)
```

3.2 Writing to CSV

```
import csv

rows = [
    {"ip": "192.168.1.1", "status": "clean"},
    {"ip": "192.168.1.2", "status": "suspicious"},
]

with open("status.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["ip", "status"])
    writer.writeheader()
    writer.writerows(rows)
```

3.3 Writing to TXT / Logs

```
with open("alerts.txt", "a") as f:
    f.write("ALERT: Malware found on host 192.168.1.5 at 14:15\n")
```

Use `logging` for structured output with timestamps:

```
import logging

logging.basicConfig(filename="alerts.log", level=logging.INFO)
logging.info("Malware detected on 192.168.1.5")
```

3.4 Writing to HTML for Simple Reports

```
html = """
<html>
  <head><title>Threat Report</title></head>
  <body>
    <h1>Incident Summary</h1>
    <p>Host 192.168.1.5 was infected with ransomware.</p>
  </body>
</html>
"""

with open("report.html", "w") as f:
    f.write(html)
```

Use this for basic web-based alerts or management reports.

---

4. Exporting Structured Reports (Real Example)

Generate a Full JSON Report from Scan Results

```
import json
import time

def create_report(ip, threats):
    report = {
        "generated_at": time.ctime(),
        "host": ip,
        "threats_detected": threats,
        "total_threats": len(threats)
```

```
    }
    return report

threats = [
    {"name": "CobaltStrike Beacon", "severity": "high"},
    {"name": "Suspicious Powershell", "severity": "medium"}
]

report = create_report("192.168.1.77", threats)

with open("scan_report.json", "w") as f:
    json.dump(report, f, indent=4)
```

---

5. Sanitizing and Validating Output

Before exporting or storing data:

- Remove sensitive fields (tokens, passwords)
- Convert non-serializable types (e.g., datetime objects)
- Sanitize filenames or paths

```
import datetime

# Convert datetime to ISO 8601 string before dumping
incident = {
    "time": datetime.datetime.now().isoformat(),
    "user": "admin",
    "event": "unauthorized_access"
}
```

```
json.dump(incident, open("incident.json", "w"))
```

---

6. Output for Integration with Other Tools

- SIEM (Splunk, ELK) expects logs in specific format (JSON or CSV)
- SOC Dashboards may read from a shared `status.json`
- Email or HTTP alerts can attach/export reports

```
# Sending report to webhook
import requests

with open("report.json") as f:
    data = json.load(f)

requests.post("https://defense.local/api/report", json=data)
```

---

7. Best Practices

- Validate all data before outputting (avoid malformed logs)
- Use timestamps and UTC consistently
- Prefer structured formats (JSON > TXT)
- Use rotating logs (`RotatingFileHandler`) to prevent huge files
- Escape special characters in CSV fields

---

8. Final Challenge

Build a Python script that:

1. Reads firewall logs from a CSV
2. Counts failed access attempts per IP
3. Generates a report of the top 5 offenders
4. Outputs to JSON, CSV, and HTML simultaneously
5. Logs a warning for each IP exceeding a threshold (e.g., > 50 failed attempts)